

AI 500

Foundations of Artificial Intelligence

L11: Neural Networks

By Murtaza Taj



Recap: Least Square Error Solution

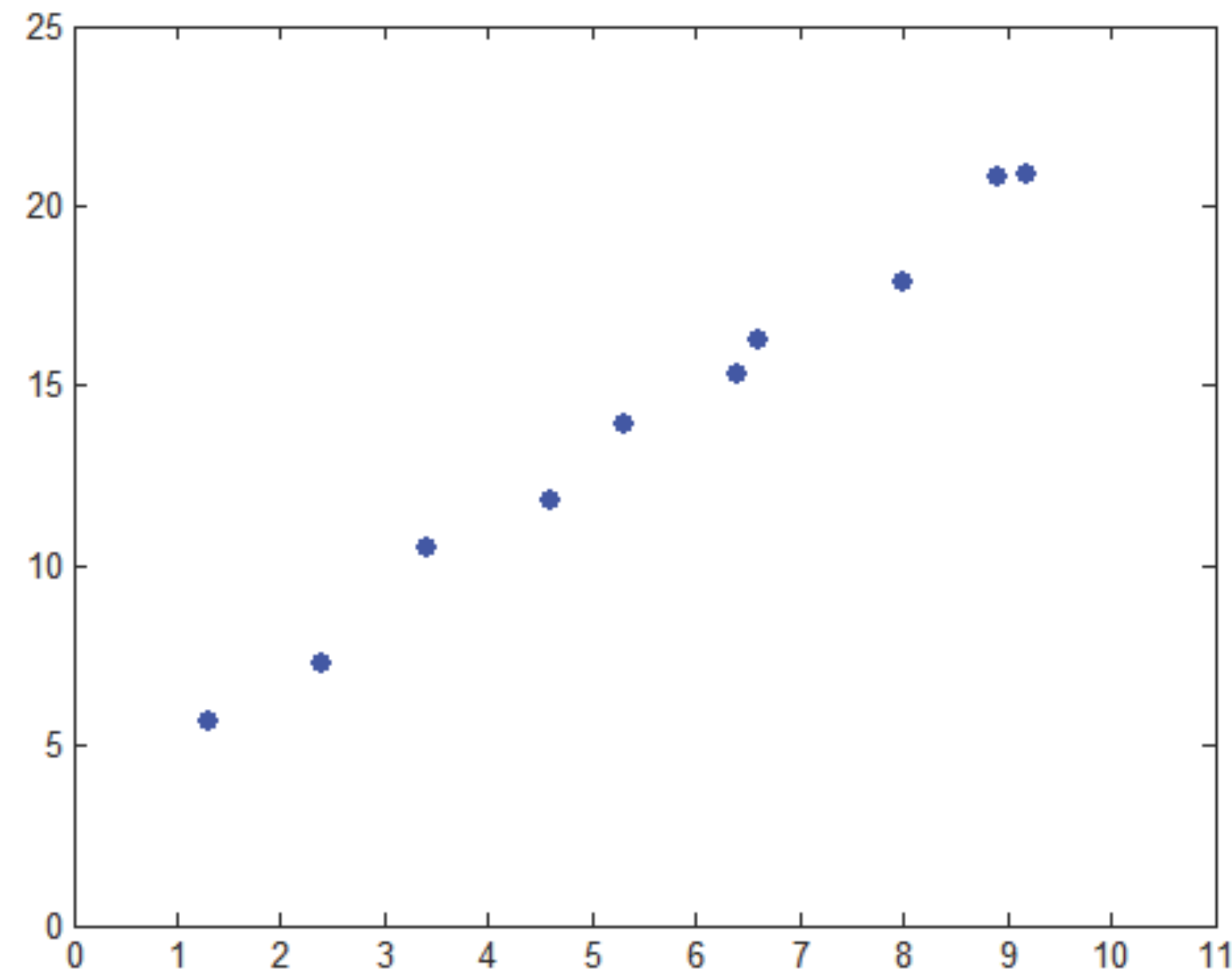
Linear Fitting

$$y = mx + c$$

$$E = \frac{1}{n} \sum_{i=1}^n (t^{(i)} - mx^{(i)} - c)^2$$

$$\frac{\partial E}{\partial m} = -\frac{2}{n} \sum_{i=1}^n (t^{(i)} - mx^{(i)} - c)x^{(i)} = 0$$

$$\frac{\partial E}{\partial c} = -\frac{2}{n} \sum_{i=1}^n (t^{(i)} - mx^{(i)} + c) = 0$$



x	t
1.3	5.7
2.4	7.3
3.4	10.5
4.6	11.8
5.3	13.9
6.6	16.3
6.4	15.3
8.0	17.9
8.9	20.8
9.2	20.9

Recap: Least Square Error Solution

Linear Fitting

$$\frac{\partial E}{\partial m} = -\frac{2}{n} \sum_{i=1}^n (t^{(i)} - mx^{(i)} - c)x^{(i)} = 0$$

$$-\sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)} = 0$$

$$-\sum_i t^{(i)}x^{(i)} + m \sum_i (x^{(i)})^2 + c \sum_i x^{(i)} = 0$$

$$\sum_i m(x^{(i)})^2 + \sum_i cx^{(i)} = \sum_i t^{(i)}x^{(i)}$$

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)}x^{(i)} \end{bmatrix}$$

$$\frac{\partial E}{\partial c} = -\frac{2}{n} \sum_{i=1}^n (t^{(i)} - mx^{(i)} + c) = 0$$

$$-\sum_i (t^{(i)} - mx^{(i)} + c) = 0$$

$$-\sum_i t^{(i)} + m \sum_i x^{(i)} + \sum_i c = 0$$

$$m \sum_i x^{(i)} + c \sum_i 1 = \sum_i t^{(i)}$$

$$\begin{bmatrix} m \sum_i x^{(i)} & c \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)} \end{bmatrix}$$

x	t
1.3	5.7
2.4	7.3
3.4	10.5
4.6	11.8
5.3	13.9
6.6	16.3
6.4	15.3
8.0	17.9
8.9	20.8
9.2	20.9

Recap: Least Square Error Solution

Linear Fitting

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \\ \sum_i x^{(i)} & \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)} x^{(i)} \\ \sum_i t^{(i)} \end{bmatrix}$$

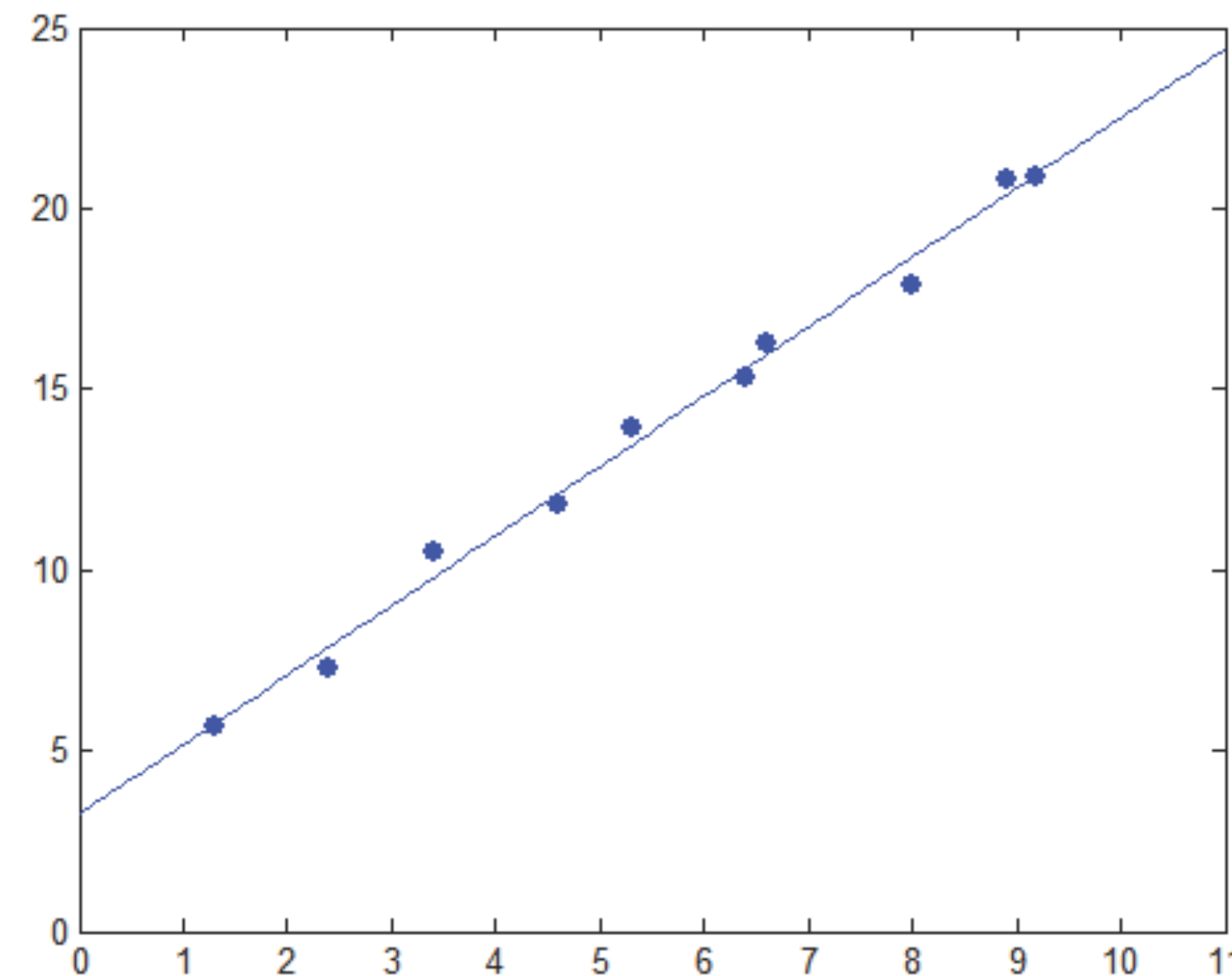
$$\begin{bmatrix} m \sum_i x^{(i)} & c \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)} \end{bmatrix}$$

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \\ \sum_i x^{(i)} & \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)} x^{(i)} \\ \sum_i t^{(i)} \end{bmatrix}$$

2×2 2×1 2×1

$$A\mathbf{x} = \mathbf{b}$$

$$\begin{bmatrix} 380.63 & 56.1 \\ 56.1 & 10 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} 914.68 \\ 140.4 \end{bmatrix}$$



<i>x</i>	<i>t</i>
1.3	5.7
2.4	7.3
3.4	10.5
4.6	11.8
5.3	13.9
6.6	16.3
6.4	15.3
8.0	17.9
8.9	20.8
9.2	20.9

Least Square Error Solution

Linear Fitting

$$y = mx + c$$

$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

$$\frac{\partial E}{\partial c} = - \sum_i (t^{(i)} - mx^{(i)} + c)$$

~~$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \\ \sum_i x^{(i)} & \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)} x^{(i)} \\ \sum_i t^{(i)} \end{bmatrix}$$

2×2 2×1 2×1~~

$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{1, x_1, x_2, \dots\}^T$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

$$\frac{\partial E}{\partial w_j} = - \sum_i (t^{(i)} - y^{(i)})x_j^{(i)}$$

Infeasible as parameters exceeds

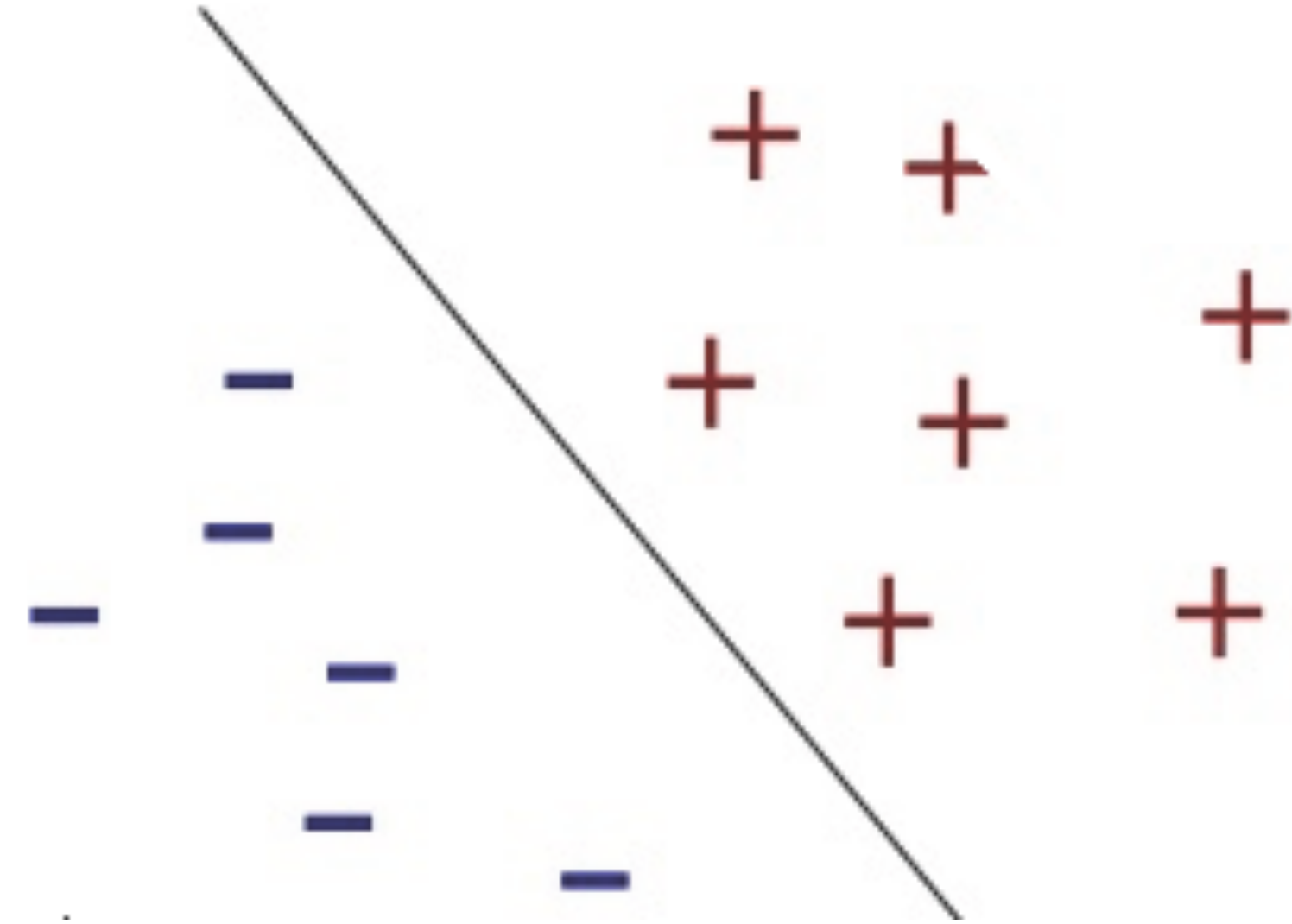
Recap: Linear Classifier

$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{1, x_1, x_2, \dots\}^T$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} = \sum_j w_j x_j^{(i)}$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$



Recap: Linear Classifier

$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{1, x_1, x_2, \dots\}^T$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} = \sum_j w_j x_j^{(i)}$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

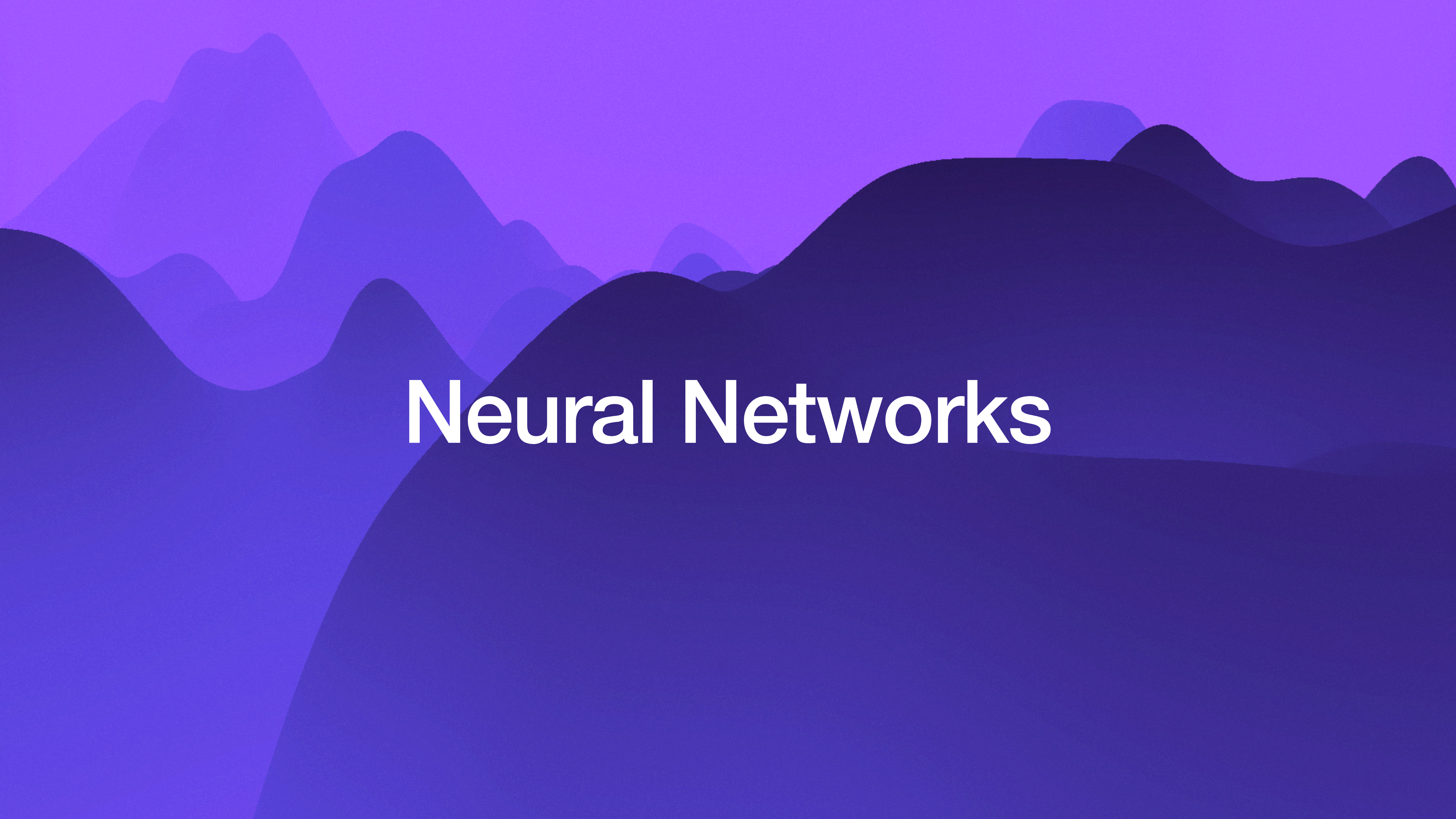
$$\frac{\partial E}{\partial w_j} = - \sum_i (t^{(i)} - y^{(i)}) x_j^{(i)}$$

Lbs	Width	Height	y
1	3	2	Insect
1	4	3	Insect
2	11	12	Bird
2	14	9	Bird

Let $\mathbf{w} = [10, -1, -1]$

$$1 \times 10 + (-1)3 + (-1)2 > 0$$

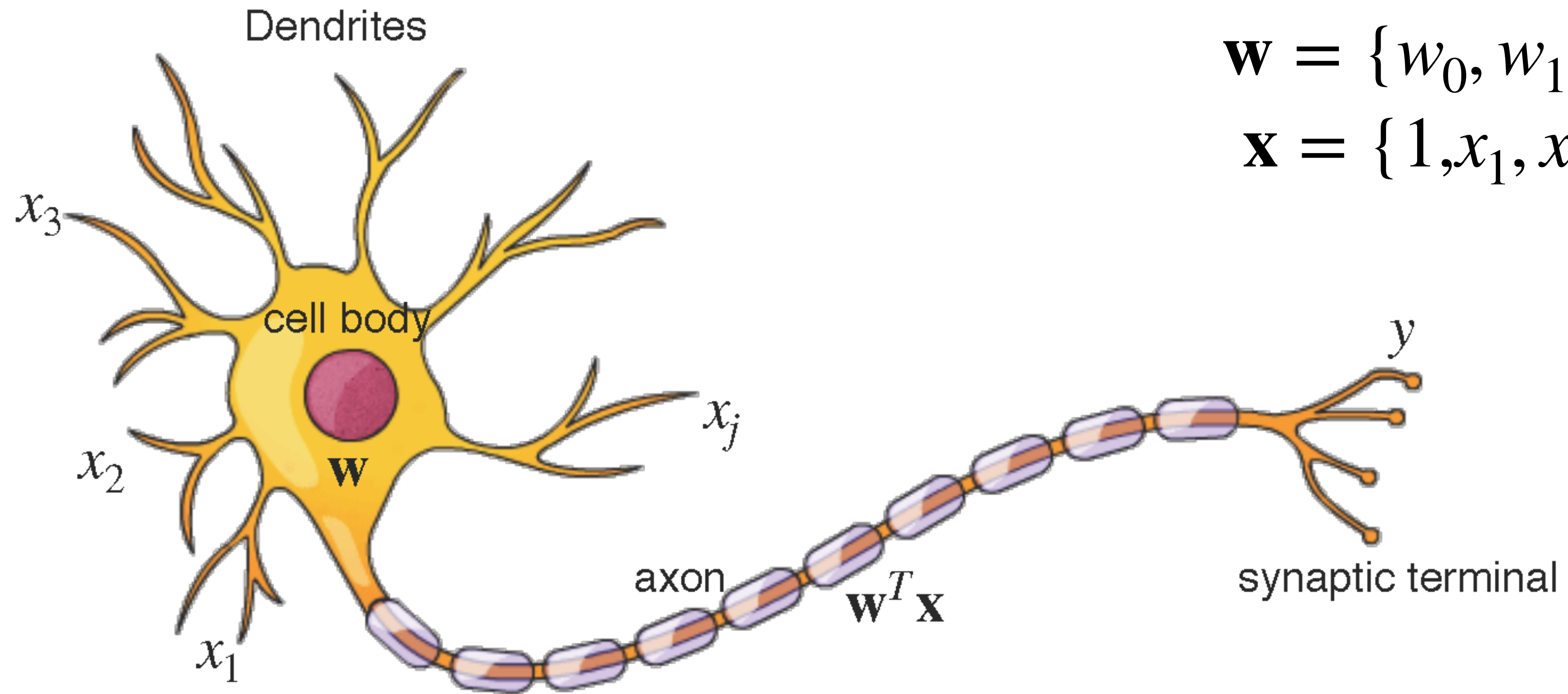
$$2 \times 10 + (-1)14 + (-1)9 \leq 0$$

The background of the image features a series of overlapping, stylized mountain ranges. The mountains are rendered in various shades of purple and blue, with the foreground mountains being a darker, more saturated blue and the background mountains being lighter, creating a sense of depth. The peaks are smooth and rounded, giving the image a soft, abstract appearance.

Neural Networks

Neuron & Linear Classifier

Biological Neuron



$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$
$$\mathbf{x} = \{1, x_1, x_2, \dots\}^T$$



Iterative Method

Error Function

$$E = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

$$\frac{\partial E}{\partial w_j} = - \sum_i (t^{(i)} - y^{(i)}) x_j^{(i)}$$

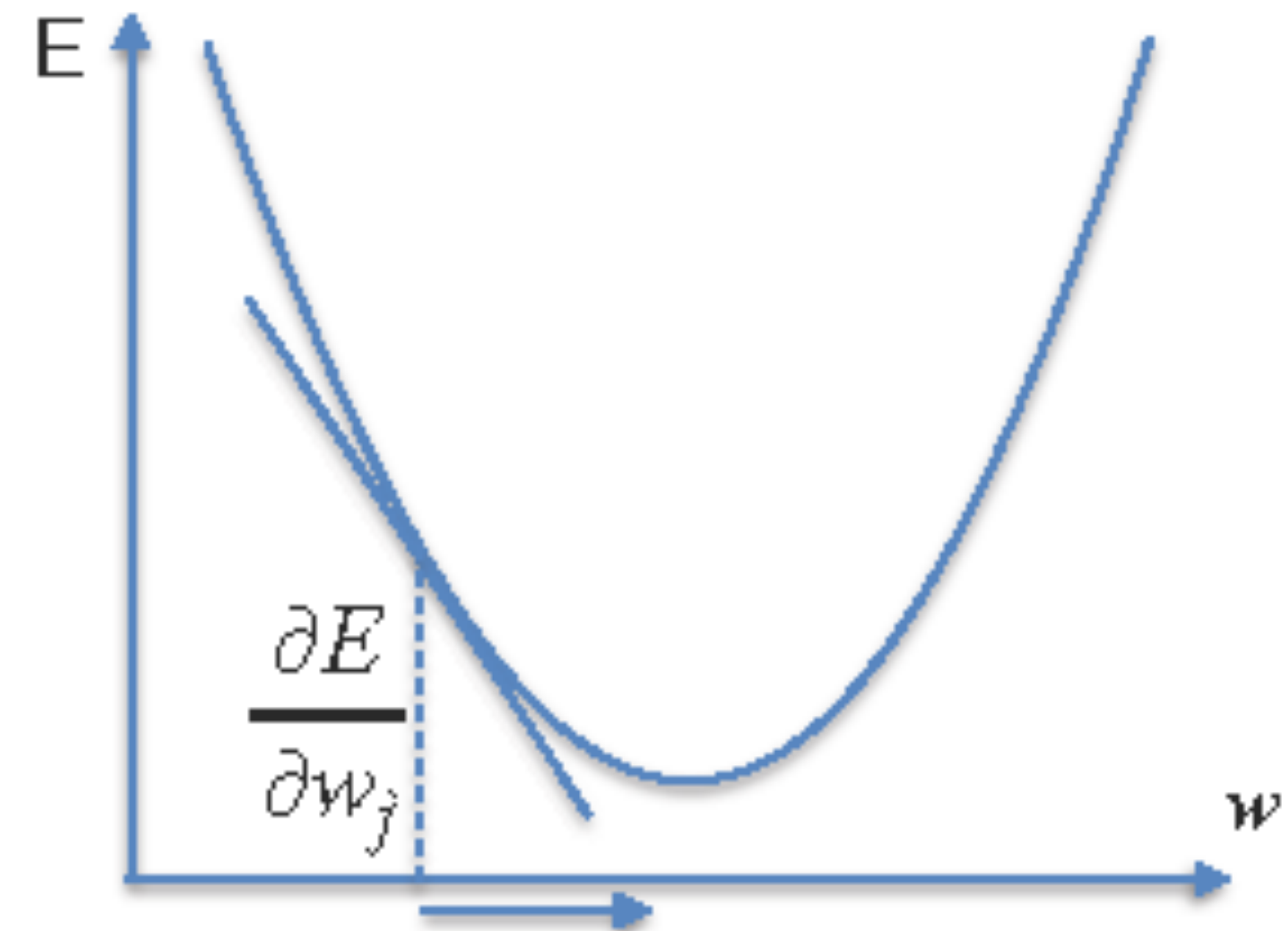
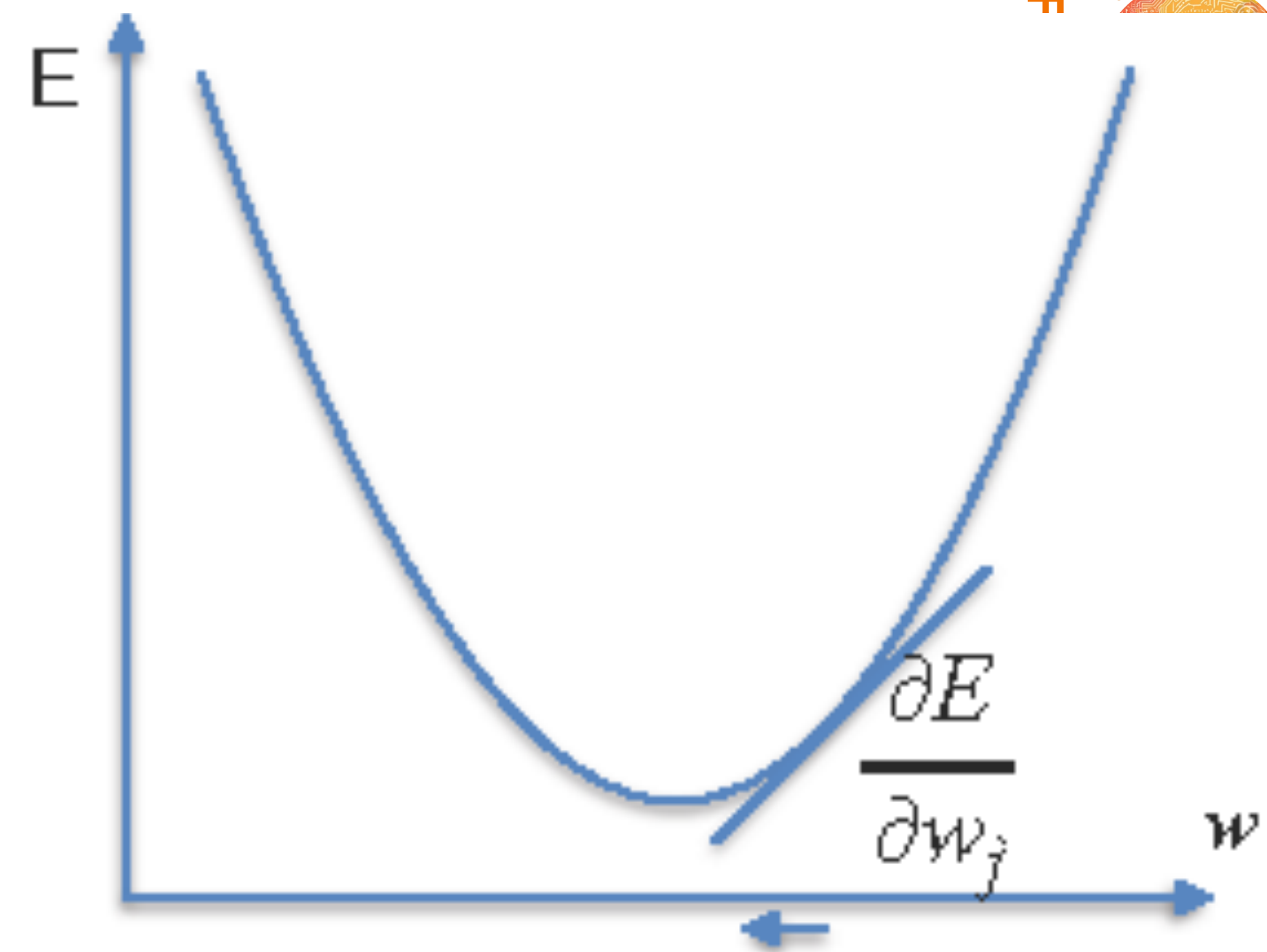
$$\frac{\partial E}{\partial w_j} = - \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)}) x_j^{(i)}$$

$$w_j = w_j - \eta \frac{\partial E}{\partial w_j}$$

$$w_j = w_j + \Delta w_j$$

$$\Delta w_j = \eta (t^{(i)} - y^{(i)}) x_j^{(i)}$$

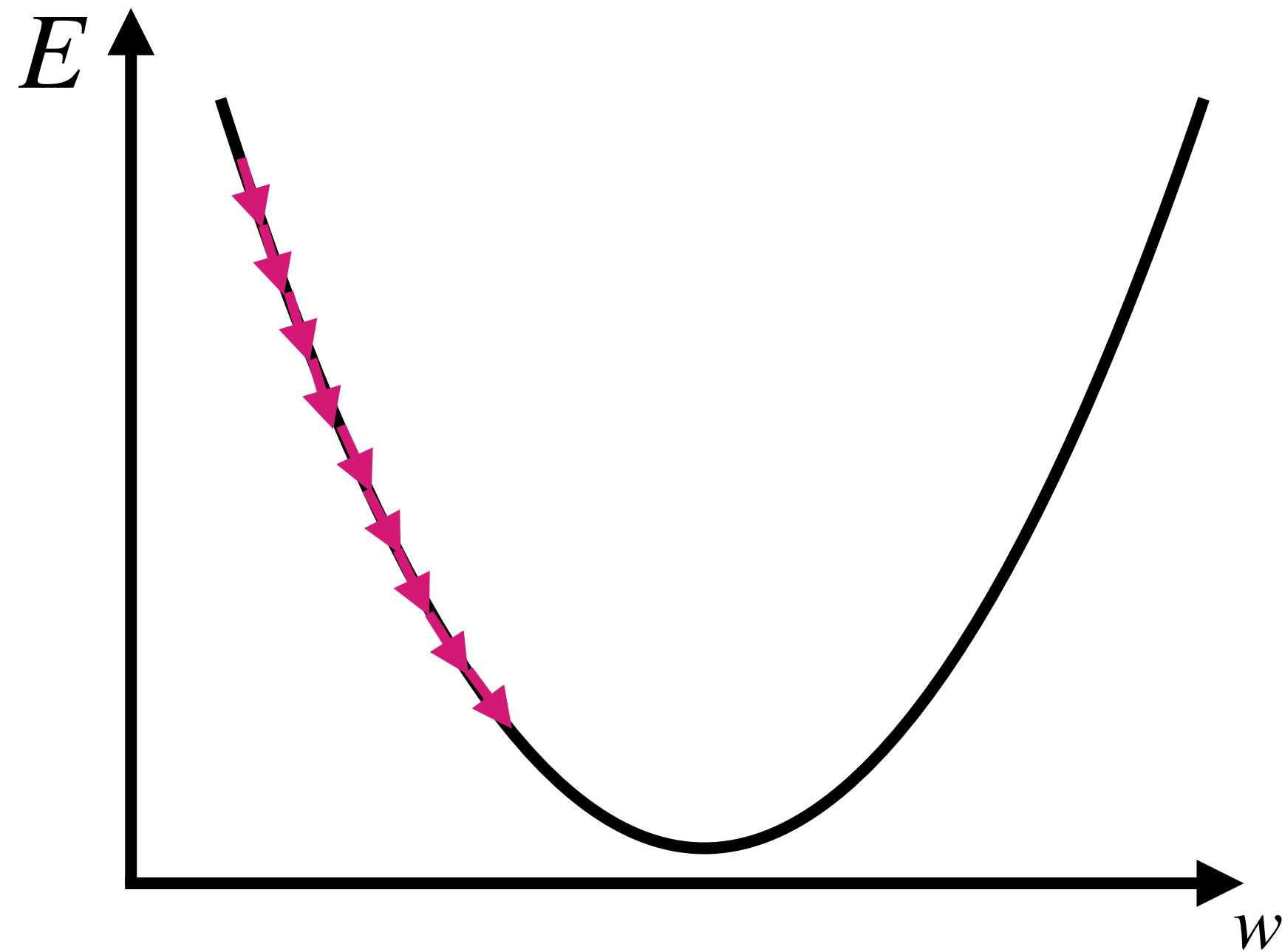
where η is the learning rate



Error Function

$$w_j = w_j - \eta \frac{\partial E}{\partial w_j}$$

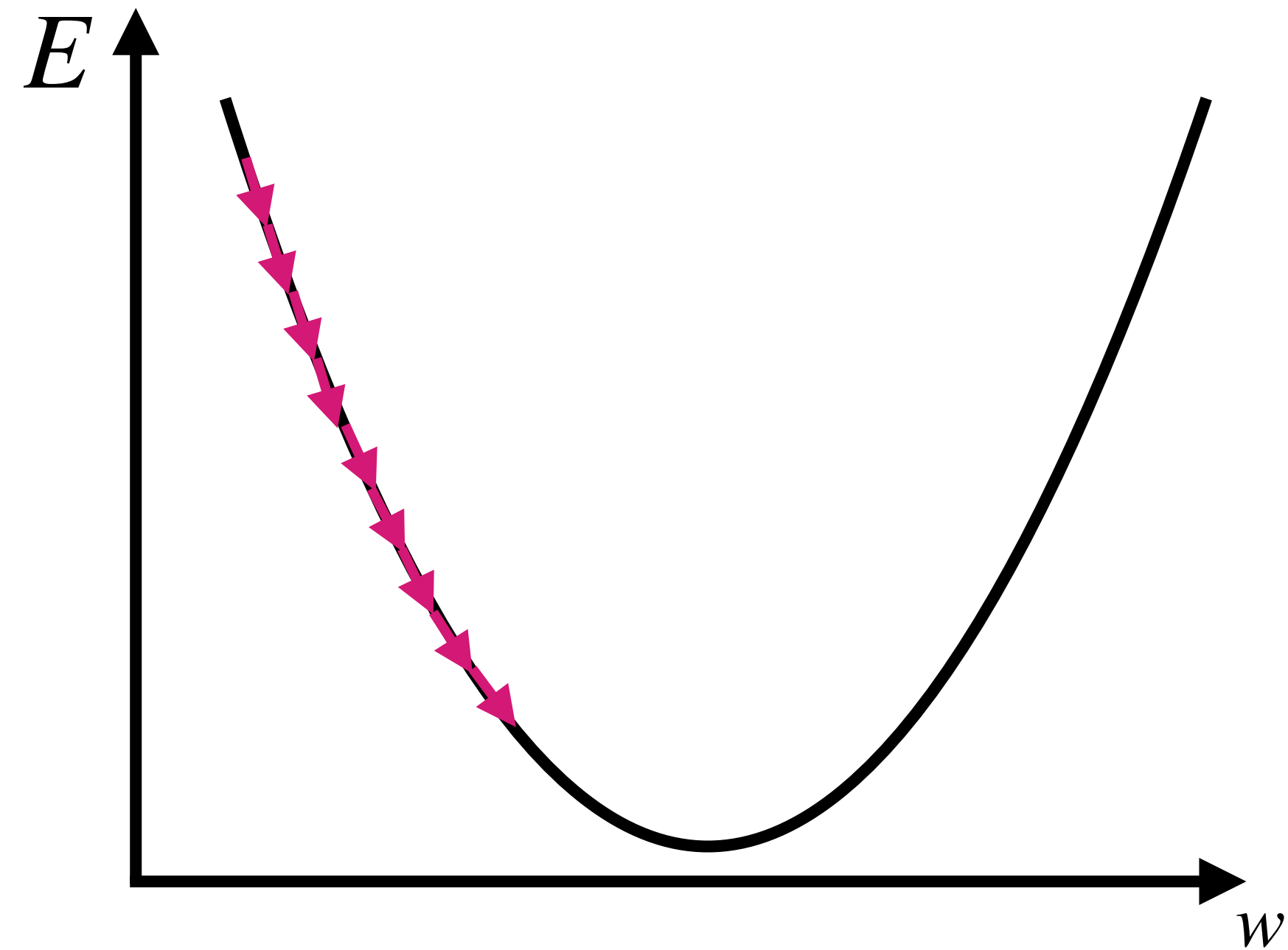
where η is too small



Error Function

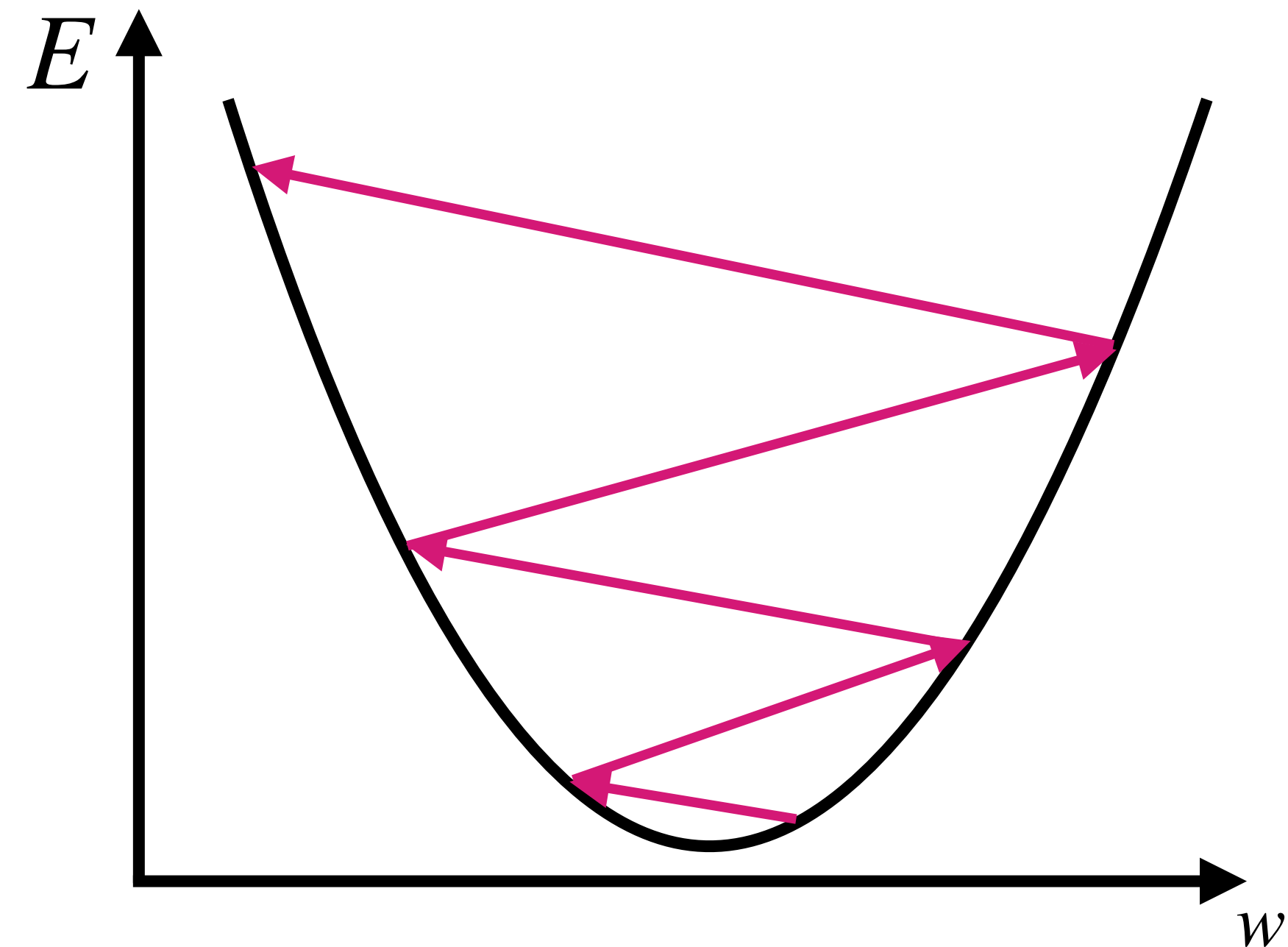
$$w_j = w_j - \eta \frac{\partial E}{\partial w_j}$$

where η is too small



$$w_j = w_j - \eta \frac{\partial E}{\partial w_j}$$

where η is too large



Why don't we solve it analytically?

- Its straightforward to write the down a set of equations, and to solve for best set of weights - so why don't we use it?
- **Scientific Answer:** We want a method that real neuron can use
- **Engineering Answer:** We want a method that generalized to multi-layer non-linear network
 - The analytical solution relies on it being linear and having a squared error measure
 - Iterative methods are usually less efficient but are much easier to generalize

Analytical vs. Iterative Solution

$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x}^{(i)} = \{1, x_1^{(i)}, x_2^{(i)}, \dots\}^T$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} = \sum_j w_j x_j^{(i)}$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

Direct/Analytical Approach

$$\frac{\partial E}{\partial w_j} = - \sum_i (t^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

Iterative Approach

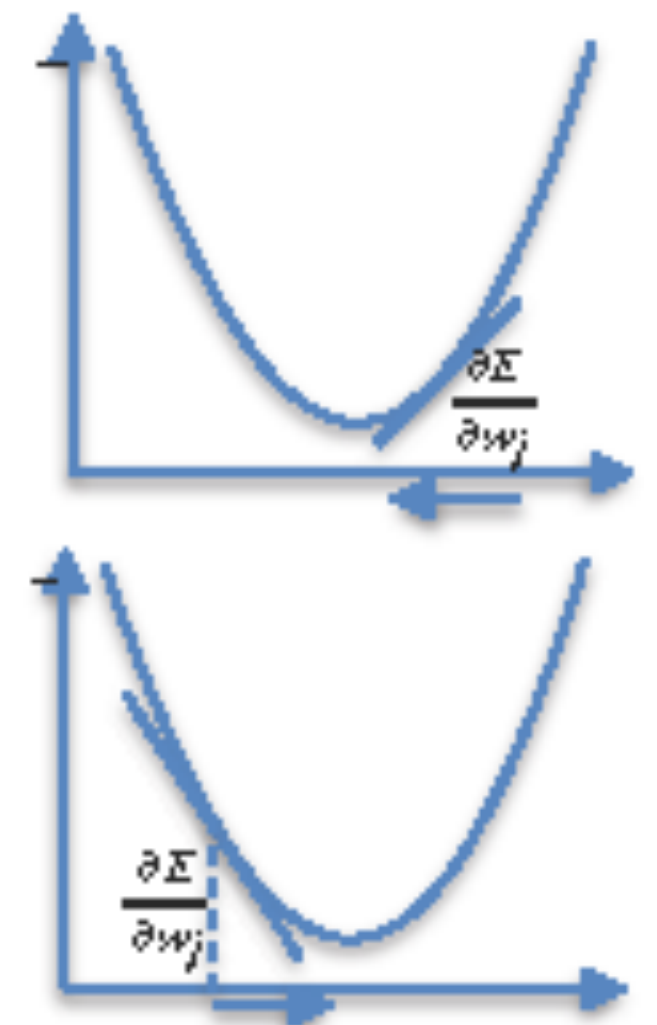
$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

$$w_j \leftarrow w_j + \Delta w_j$$

$$\Delta w_j = - \eta \frac{\partial E}{\partial w_j}$$

$$\Delta w_j = \eta (t^{(i)} - y^{(i)}) x_j^{(i)}$$

where η is the learning rate



Iterative Method Example

Iterative Method

A Toy Example to Illustrate the Iterative Method

- Each day you get the **lunch** at the cafeteria
 - Your diet consists of **fish**, **chips** and **ketchup**
 - You get several portions of each
- The cashier only tells you the **total price of the meal**
 - After several days you should be able to figure out the **price of each portion**
- **The iterative approach:** Start with random guesses for the price and then adjust them to get a better fit to the observed prices of whole meals

Iterative Method

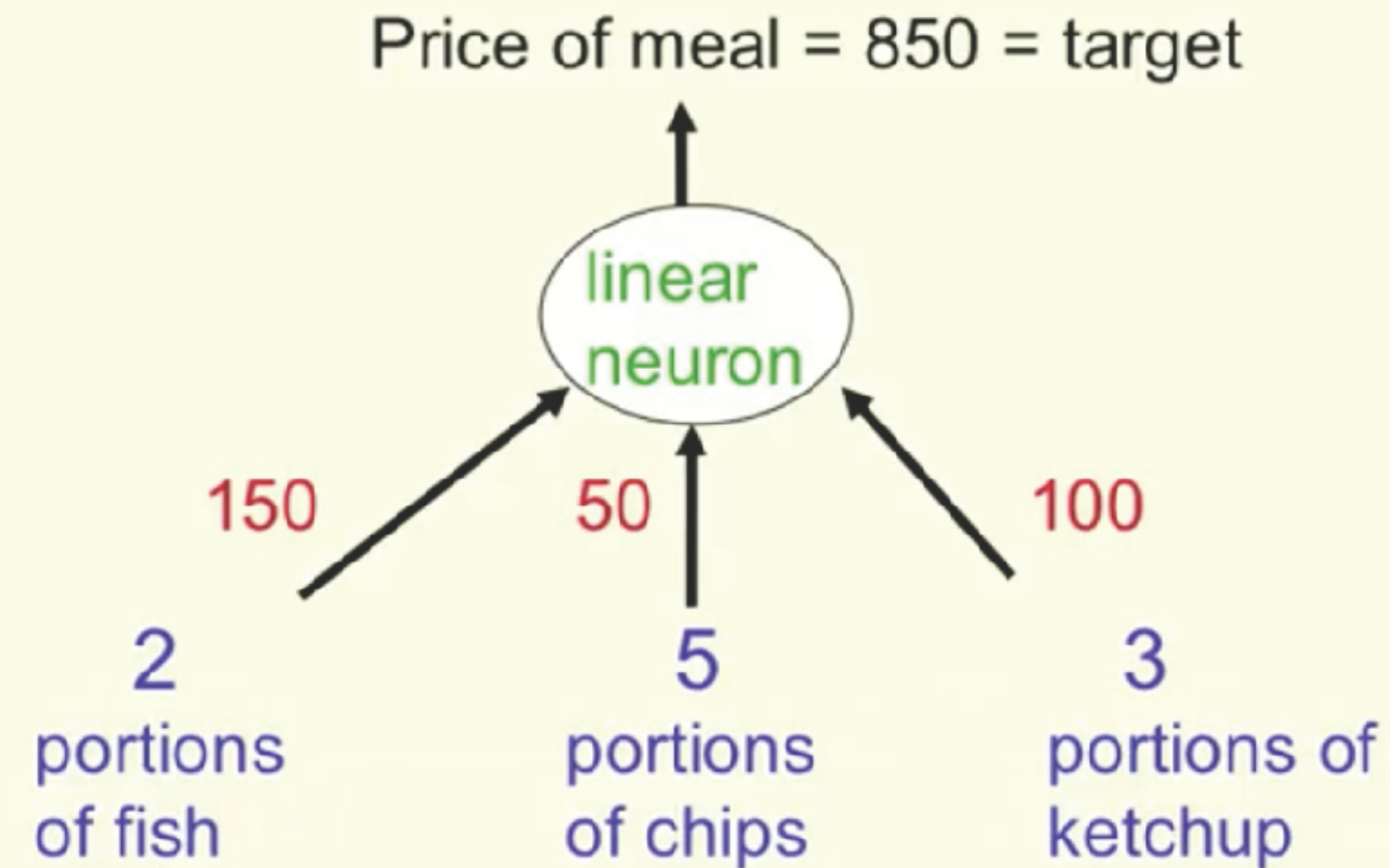
Solving the equation iteratively

- Each meal price gives a linear constrain on the prices of the portions

Iterative Method

Solving the equation iteratively

The **true** weights used by the cashier



Iterative Method

Solving the equation iteratively

- Each meal price gives a linear constrain on the prices of the portions

$$\text{price} = x_{fish}w_{fish} + x_{chips}w_{chips} + x_{ketchup}w_{ketchup}$$

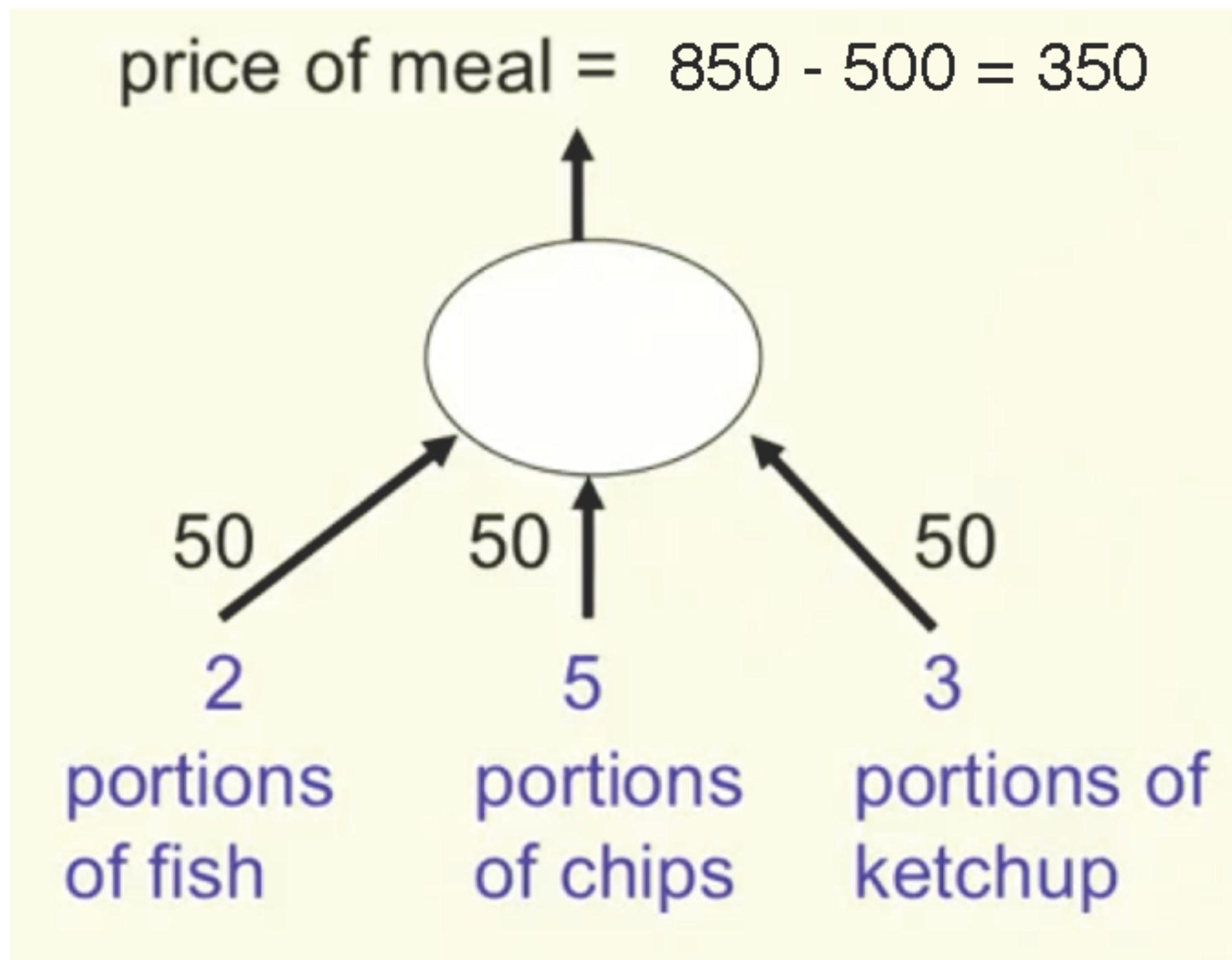
- The prices of the portions are like the weights of a linear neuron

$$\mathbf{w} = \{w_{fish}, w_{chips}, w_{ketchup}\}$$

- We will start with guesses for the weights and then adjust the guesses slightly to give a better fit to the prices given by the cashier

Iterative Method

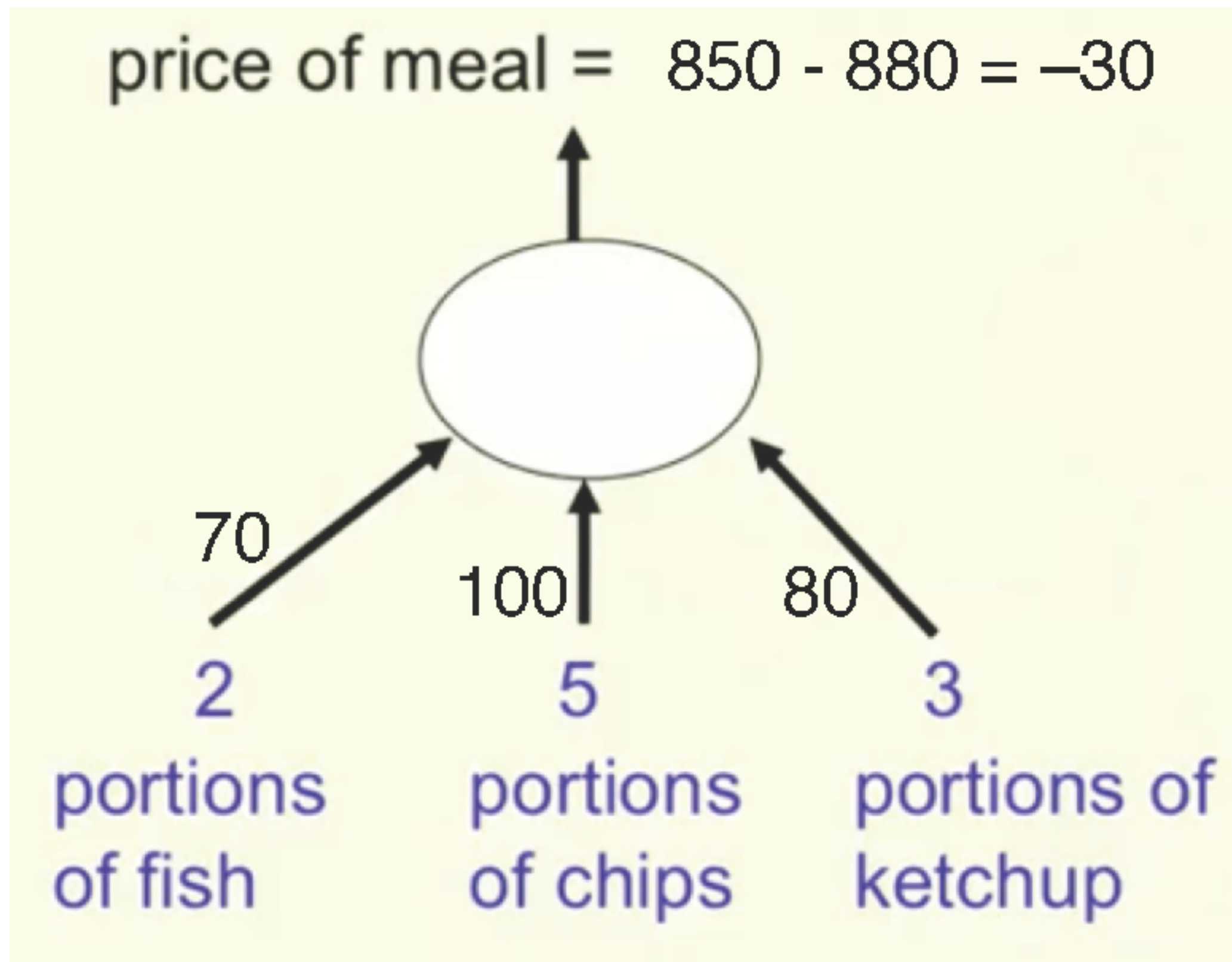
A model of the cashier with arbitrary initial weights



- Residual error = 350
- The “delta-rule” for learning is $\Delta w_j = \eta(t^{(i)} - y^{(i)})x_j^{(i)}$
- With a learning rate η of $1/35$, the weight changes are +20, +50, +30
- This gives the new weights of 70, 100, 80
- Notice that the weights for chips got worse!

Iterative Method

A model of the cashier with arbitrary initial weights



- Residual error = -30
- The “delta-rule” for learning is $\Delta w_j = \eta(t^{(i)} - y^{(i)})x_j^{(i)}$
- With a learning rate η of 1/35, the weight changes are -1.71, -4.29, -2.57
- This gives the new weights of 68.29, 95.71, 77.43

Iterative Method Behaviors

- Does the learning procedure eventually get the **right answer**
 - There may be **no perfect answer**
 - By making the learning rate small enough we can **get as close as we desire** to the best answer
- How quickly do the weights **converge** to their correct values?
 - It **can be very slow** if two input dimensions are highly correlated

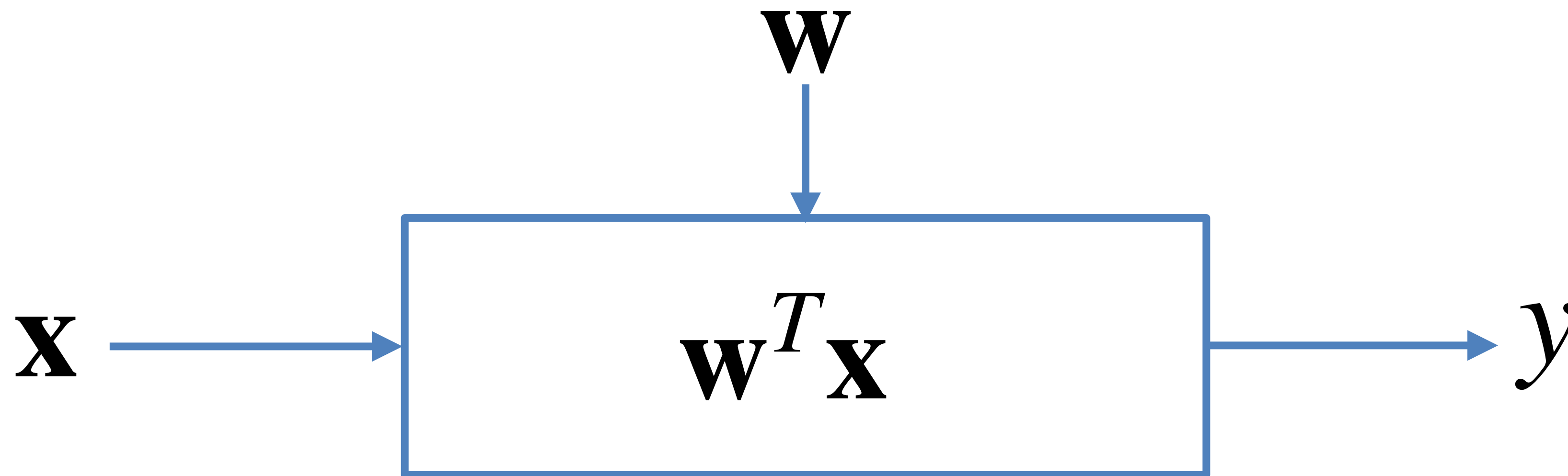
Try it yourself

- Dry run it both analytically as well as iteratively and see if you get 150, 50 and 100 as price for fish, chips and ketchup respectively

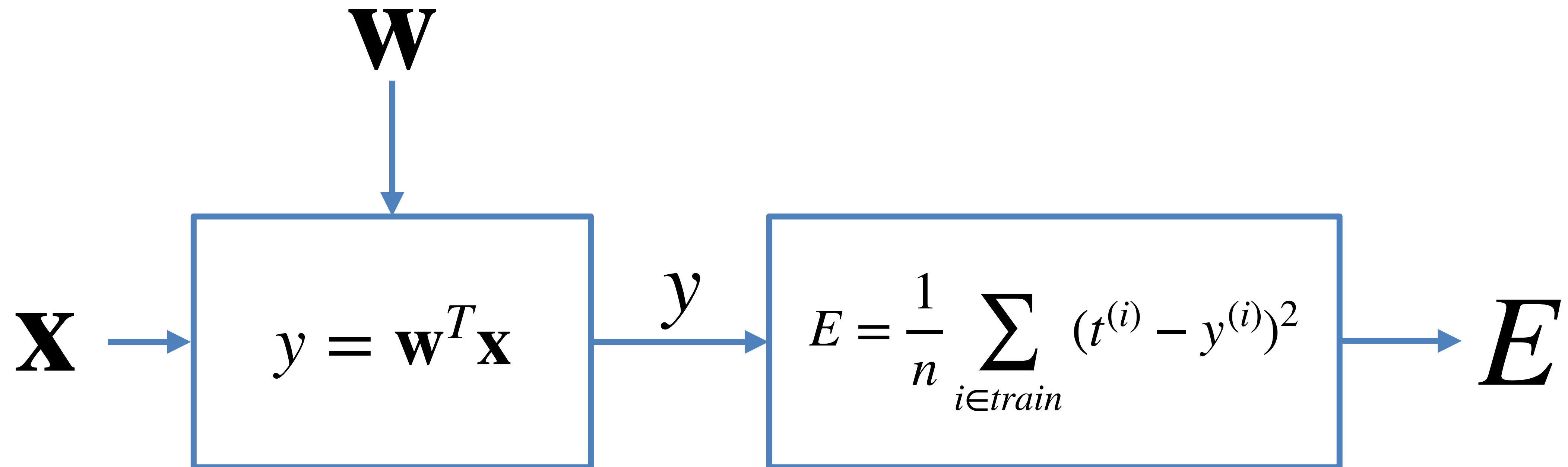
No. Fish ($x^{(1)}$)	2	1	4	2	1
No. Chips ($x^{(2)}$)	5	3	1	4	3
No. Ketchup ($x^{(3)}$)	3	4	3	1	5
Total Price (y)	850	700	950	600	800

Key Computation: Forward Pass

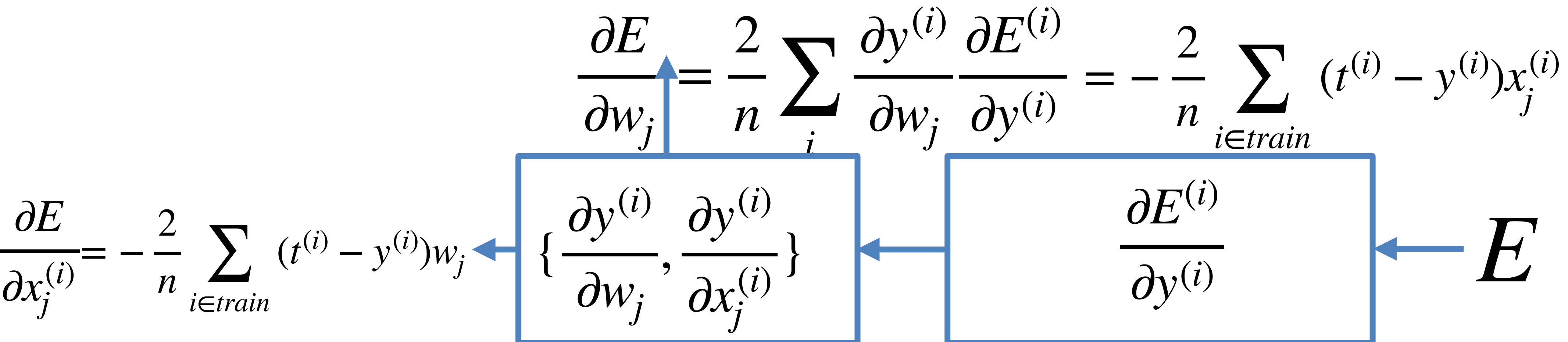
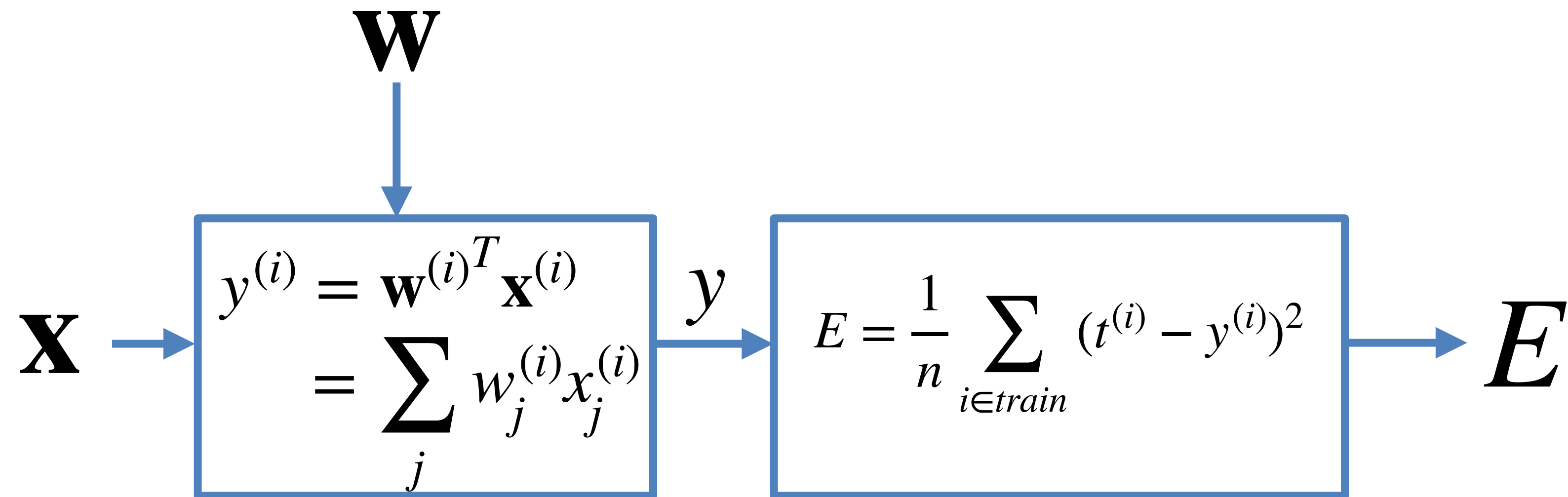
$$y = \mathbf{w}^T \mathbf{x}$$



Key Computation: Forward Pass



Key Computation: Forward/Backward Pass



Gradient Descent

- $\Delta w_j \leftarrow \text{Initialize to ZEROS}$
- For each iteration
 - For each training example $(\mathbf{x}^{(i)}, t^{(i)})$
 - For each weight w_j

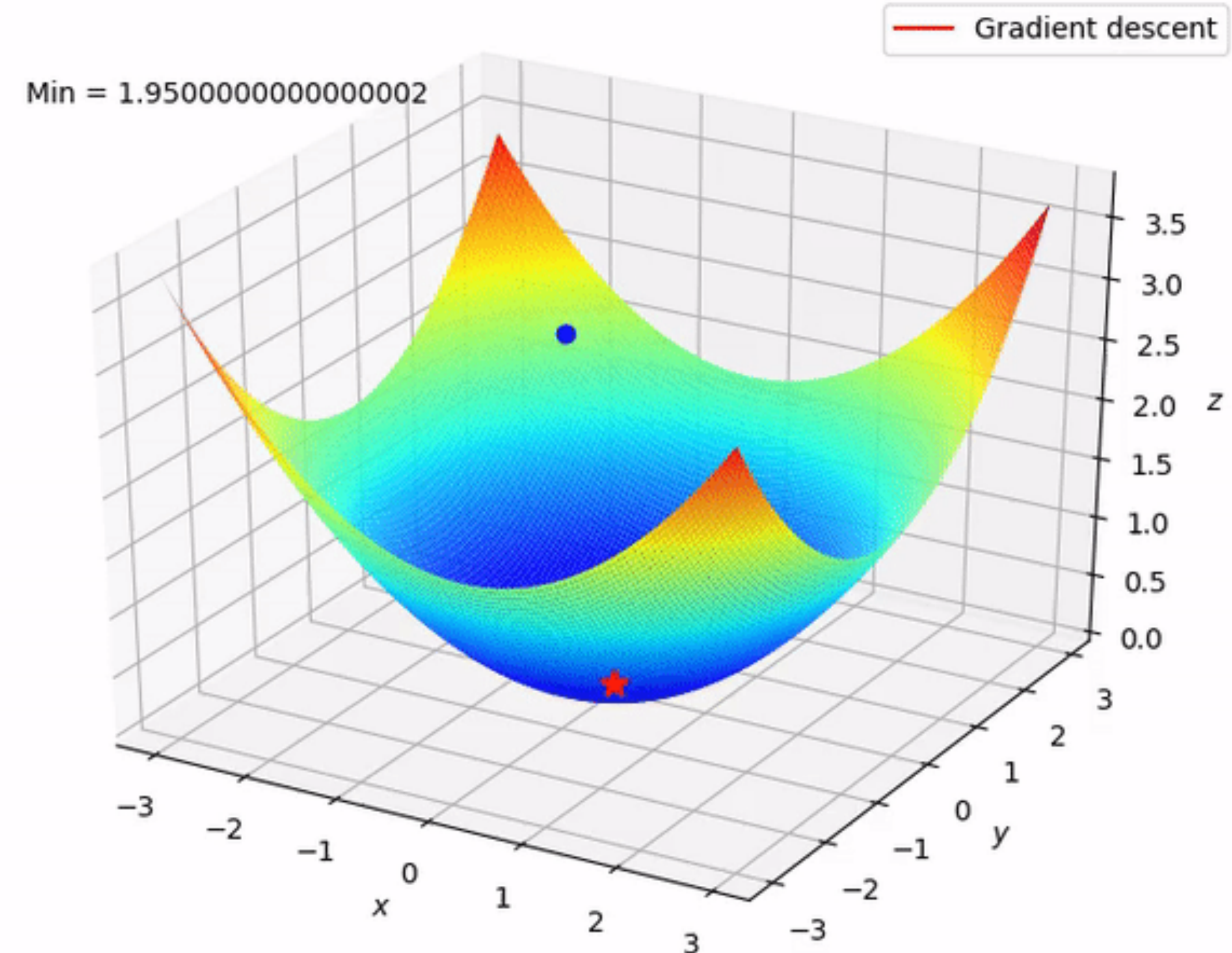
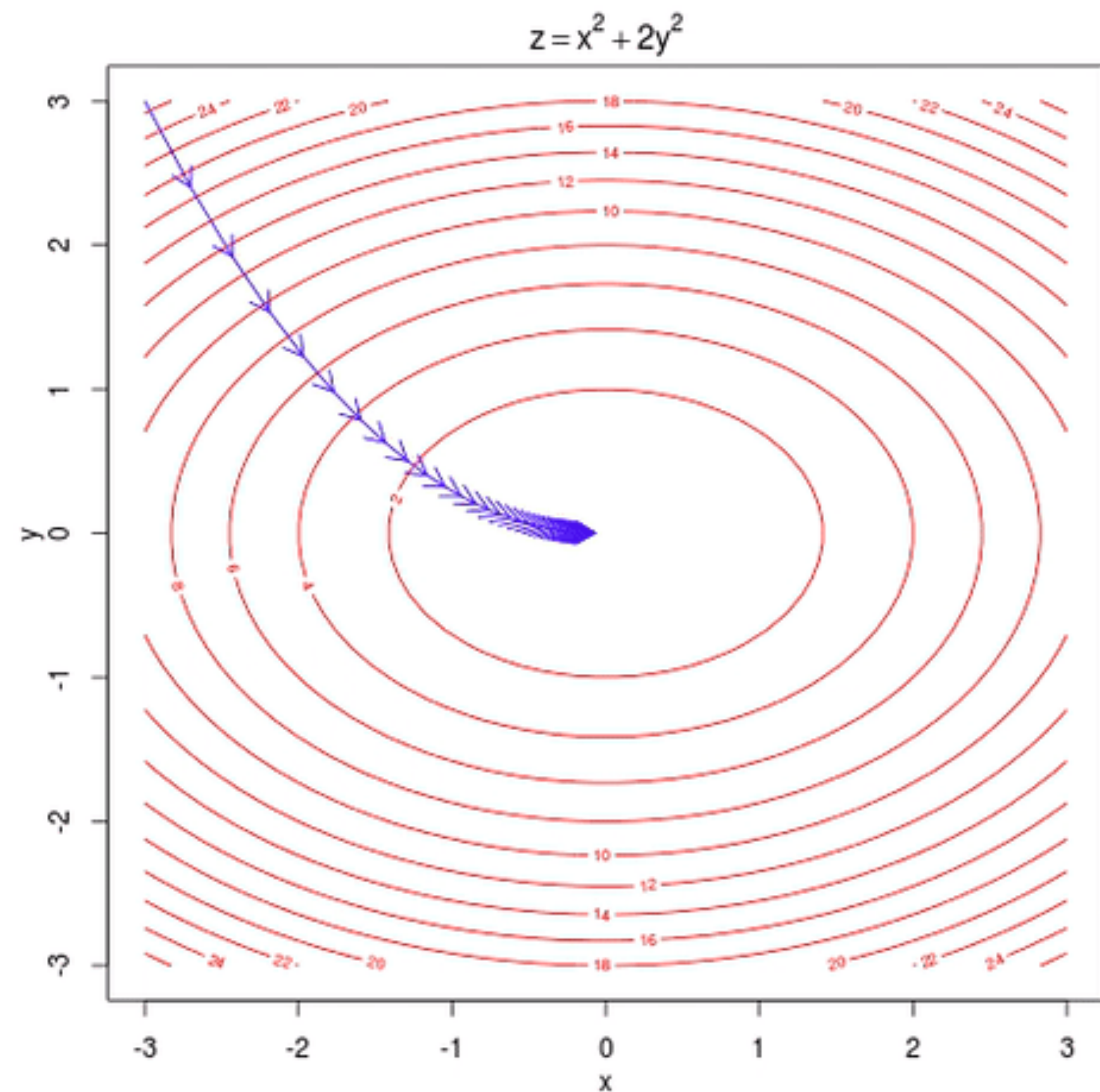
$$\Delta w_j \leftarrow \Delta w_j + \eta x_j^{(i)} (t^{(i)} - y^{(i)})$$

- For each weight w_j

$$w_j \leftarrow w_j + \Delta w_j$$

$$\Delta w_j = \eta \sum_i x_j^{(i)} (t^{(i)} - y^{(i)})$$

Gradient Descent



Gradient Descent - Variants

- Vanilla/Batch Gradient descent:
 - Uses all samples per update
- Stochastic Gradient Descent:
 - Uses only one sample per update
- Mini-batch Gradient Descent:
 - Uses batch of samples

$$\Delta w_j = \eta \sum_i x_j^{(i)} (t^{(i)} - y^{(i)})$$

$$w_j \leftarrow w_j + \Delta w_j$$

$$\Delta w_j = \eta x_j^{(i)} (t^{(i)} - y^{(i)})$$

$$w_j \leftarrow w_j + \Delta w_j$$

$$\Delta w_j = \eta \sum_{k \in \text{mini-batch}} x_j^{(k)} (t^{(k)} - y^{(k)})$$

$$w_j \leftarrow w_j + \Delta w_j$$